소속 변경과 발주 승인

업무 설명용 합성 시나리오입니다. 판정과 함께 적용한 규칙, 사용한 근거, 재검토 대상을 제시하는 것이 목표입니다.

"검토자의 소속 본부가 바뀌었습니다. 이 발주를 그대로 승인해도 되나요?"

구매 담당자

검토자 소속이 바뀌었는데
이 발주를 진행해도 될까?

승인 담당자

자격과 소속 관계에
바뀐 규정도 적용됐을까?

감사 담당자

어떤 기록과 규정 버전으로
그렇게 판단했을까?

발표 구성

업무의 질문

왜 이 판단이 필요한가
어떤 근거를 설명해야 하나

작은 부품 예제

표현 → 추론 → 검사
각 도구의 역할 구분

우리 연구의 설계

기존 결과와 남은 질문
공정한 비교와 사례 구성

- **업무 질문:** 판단에 필요한 데이터와 의미
- **제조 부품 예제:** 표현·추론·검증의 차이
- **연구 결과:** 검사한 입력·조건·출력
- **후속 설계:** 실제 자료의 한계와 데모 화면

데이터의 의미·연결·시점

> **근거**는 판단에 사용한 기록이고 **시맨틱**은 그 기록의 뜻과 관계입니다. 같은 status라도 계약·승인·자격에서 의미가 다릅니다.



- 키로 연결: 부품 P1 → 계약 C7 → 검토자 R2 → 소속 이력
- 시점 확인: 승인 당시 소속과 현재 소속이 다르면 어느 규칙을 적용할지 결정

설명용 합성 레코드 · 이력은 개념 예시이며 실제 실행 입력은 한 시점의 스냅샷

규칙 변경의 담당자와 요구사항

> 기존 Python·SQL·룰 엔진·DAG로도 구현할 수 있습니다. 연구의 관심은 변경된 의미와 규칙을 여러 소비처에 일관되게 전달하는 방법입니다.

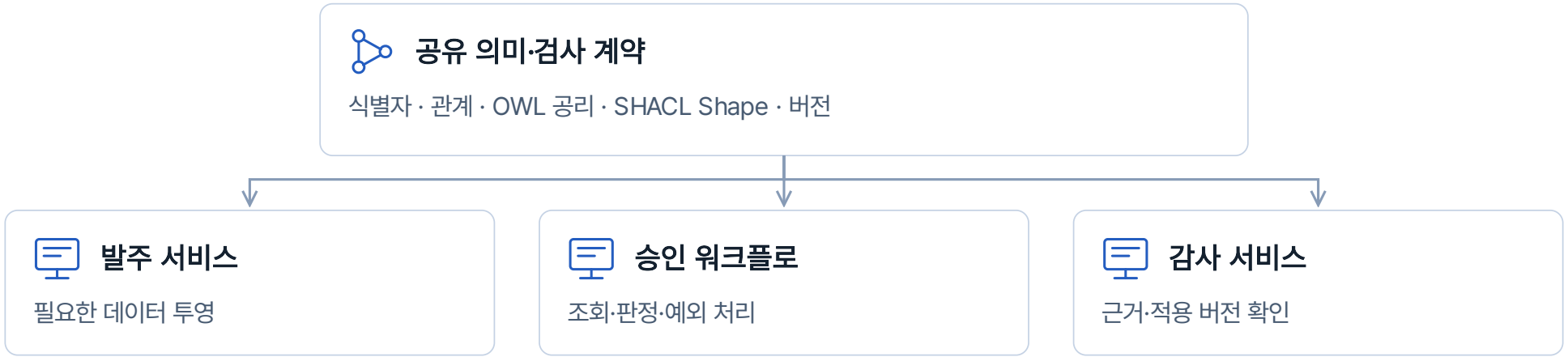
담당자별로 다른 변경 판단 근거



- 공유 JSON·스키마 레지스트리·공통 라이브러리도 비교 대상
- 표준 형식 도입만으로 변경 비용과 배포 문제가 사라지지는 않음

공유 의미와 검사 계약

> 연구 의도는 의미와 검사 조건을 공통 자산으로 관리하는 것입니다. 매핑, 버전 배포, 접근 제어와 회귀 테스트는 별도로 구현해야 합니다.



구성	공유하려는 내용
RDF	개체의 식별자와 관계 표현
OWL	클래스·관계의 공리와 함의의 기준
SHACL / Shape	검사 대상과 조건을 담은 규칙 구조
pySHACL	Shape를 실행해 표준 검증 보고서 생성

출처: RDF · OWL · SHACL · pySHACL

추론·검증·사실 확인의 구분

> 입력이 틀리면 논리적으로 일관된 결론도 현실에서는 틀릴 수 있습니다.

추론

정의와 기록으로부터
어떤 결론이 따라오는가

→

데이터 검증

제출한 데이터가
정해 둔 조건을 만족하는가

→

현실의 사실 확인

원천 근거·현장·전문가로
입력과 결론을 확인

작업	묻는 질문	필요한 근거
추론	정의와 기록에서 어떤 결론이 따라오는가	입력 사실 + 공리·규칙
데이터 검증	제출한 데이터가 조건을 만족하는가	검사할 그래프 + Shape
현실의 사실 확인	기록과 결론이 실제 업무에서 맞는가	독립 자료·현장 확인·전문가 판단

RDF·RDFS·OWL·SHACL의 역할

> 언어·정의와 실행 도구를 구분합니다. 파일을 읽는 것만으로 추론이 켜지지는 않습니다.

구성	역할	부품 P1 예
RDF	개체·관계를 트리플로 표현	P1은 조립품 A1에 장착된다
RDFS	클래스 포함 관계, domain/range 등 정의	볼트는 부품, 부품은 구성 부품
OWL	더 풍부한 논리적 공리 표현	역관계, 클래스 조합·배타성, 동일성
추론기	지원하는 공리·규칙의 결론 계산	P1도 구성 부품이라는 타입 도출
SHACL	주어진 RDF의 조건 충족 검사	제조사 식별자가 기록됐는가

출처: RDF · RDFS · OWL · SHACL

RDF 트리플과 제조 부품

> RDF는 개체와 관계를 식별자로 연결합니다. Turtle은 그 그래프를 텍스트로 적는 형식입니다.

관계 하나를 주어 · 술어 · 목적어로 기록



ex: = https://example.org/manufacturing/

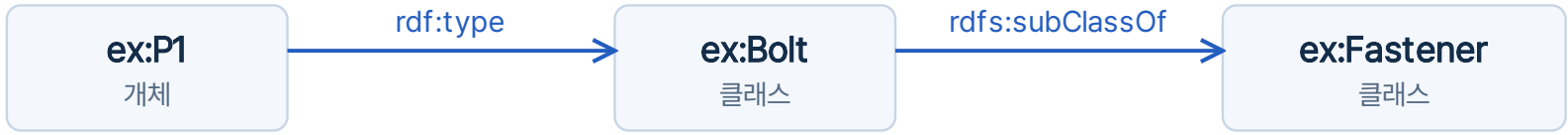
```
@prefix ex: <https://example.org/manufacturing/> .
ex:P1 a ex:Bolt ;                # P1은 볼트 개체
    ex:installedIn ex:A1 .       # P1은 조립품 A1에 장착
```

- 입력은 설명용 합성 부품 기록이며 실제 설치 사실 확인과는 별개

개체·클래스·포함 관계

> P1은 하나의 부품이고 Bolt는 그 부품이 속한 클래스입니다. 클래스 간 포함 관계와 개체의 소속을 구분합니다.

개체의 타입과 클래스의 포함 관계



rdf:type은 개체 → 클래스, rdfs:subClassOf는 클래스 → 클래스 관계다.

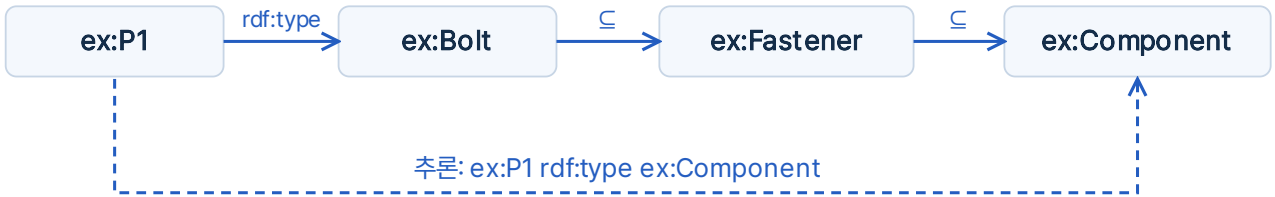
```
ex:P1 a ex:Bolt .                          # 개체 → 클래스
ex:Bolt rdfs:subClassOf ex:Fastener .      # 클래스 → 상위 클래스
```

출처: RDFS 클래스와 인스턴스

RDFS 클래스 계층과 타입 추론

> 하위 클래스의 개체는 상위 클래스에도 속합니다. 점선은 정의를 적용해 얻는 추가 결론입니다.

클래스 포함 관계에서 개체 타입 도출



⊆ = rdfs:subClassOf · 실선은 명시한 관계, 점선은 규칙으로 도출한 관계

- 입력: `P1 a Bolt` + `Bolt ⊆ Fastener ⊆ Component`
- 출력: `P1 a Fastener`, `P1 a Component`
- 실제 부품의 정체가 맞는지는 원천 자료로 별도 확인

출처: RDFS

domain·range의 타입 해석

> domain과 range는 관계의 주어·목적어를 어떤 타입으로 해석할지 정합니다. 입력을 거부하는 검증 규칙과는 다릅니다.

domain · range는 관계 양끝의 타입을 도출



타입 추론 규칙이다. 허용 타입을 검사하는 입력 검증 규칙과 구분한다.

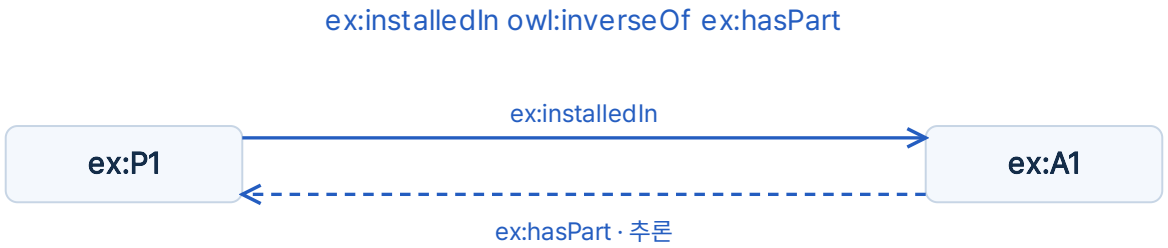
```
ex:installedIn rdfs:domain ex:Component ; # 주어는 구성 부품
               rdfs:range ex:Assembly . # 목적어는 조립품
```

출처: RDFS domain/range

OWL 공리와 역관계

> OWL의 공리는 관계와 클래스의 논리적 의미를 명시합니다. 실제 계산 범위는 선택한 추론기의 지원 기능에 달려 있습니다.

역속성 정의로 반대 방향 관계 도출



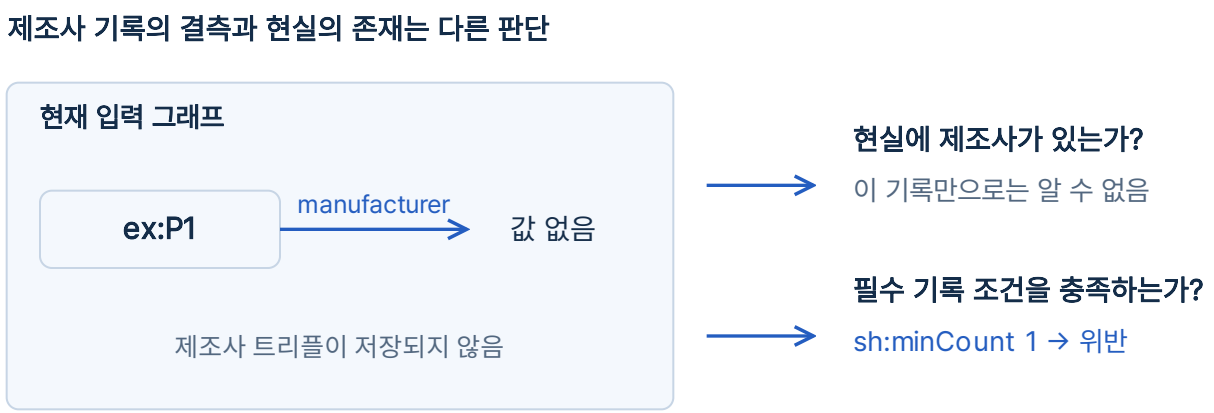
역방향 관계는 owl:inverseOf 선언이 있을 때 도출한다.

- `inverseOf` : P1이 A1에 장착되면 A1은 P1을 포함
- `equivalentClass` : “인증 부품이면서 체결 부품” 같은 분류 정의
- `disjointWith` / `sameAs` : 배타 분류 / 동일 개체 선언

출처: OWL 2 Primer

열린 세계와 기록 부재

> 제조사 기록이 없다는 사실만으로 제조사가 존재하지 않는다고 결론 내릴 수 없습니다. 제출 자료의 필수 항목 검사는 별도로 선언합니다.



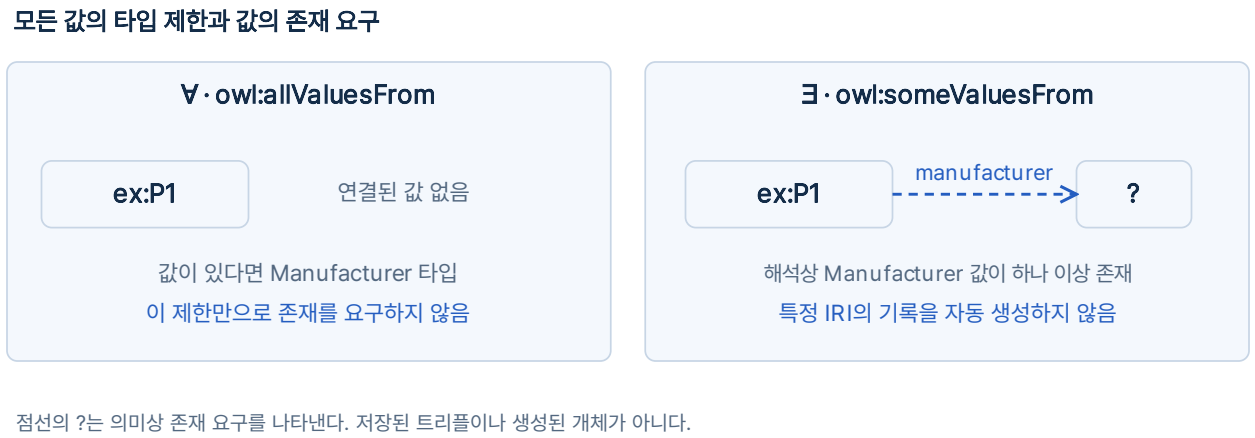
OWL의 열린 세계 해석과 SHACL의 입력 그래프 검사를 구분한다.

- OWL: 추가 근거가 없으면 제조사에 대한 판단을 유보
- SHACL: 제출된 P1 기록에 제조사 IRI가 없으면 `minCount` 위반

출처: OWL 열린 세계 가정

전체 조건·존재 조건·동일성

> “모두 인증 부품”이라는 조건은 연결 대상의 존재를 보장하지 않습니다. 식별자 두 개도 서로 다른 실체 둘을 자동으로 뜻하지 않습니다.



점선의 ?는 의미상 존재 요구를 나타낸다. 저장된 트리플이나 생성된 개체가 아니다.

- `allValuesFrom`: 연결된 대상이 있다면 지정 클래스에 속함
- `someValuesFrom`: 지정 클래스에 속한 대상의 존재를 요구
- 다른 회사임을 명시하려면 별도 동일성·차이 선언과 근거 검토

출처: OWL 제약과 동일성

추론으로 추가되는 조회 결과

추론을 쓰는 목적: 직접 입력하지 않은 결론도 정의를 따라 찾아내는 것

기록과 정의

입력 기록

P1은 볼트다.
P1은 조립품 A1에 장착된다.

함께 제공한 정의

볼트 ⊆ 체결 부품 ⊆ 구성 부품
장착된다 ↔ 포함한다 (역관계)

추론 후 추가로 조회할 수 있는 결론

P1은 구성 부품이다.

A1은 P1을 포함한다.

직접 적지 않은 타입과 역방향 관계를
입력과 정의를 결합해 계산한다.

- 다음 장에서는 같은 입력에 RDFS·OWL-RL을 적용한 실제 조회 결과를 비교한다.

원본·RDFS·OWL-RL의 조회 결과

> 같은 입력이라도 적용한 정의와 조회 범위에 따라 보이는 결론이 달라집니다. 제조사 기록은 세 조건 모두 비어 있습니다.

저장된 조회 결과 · 없음은 현실의 거짓을 뜻하지 않음

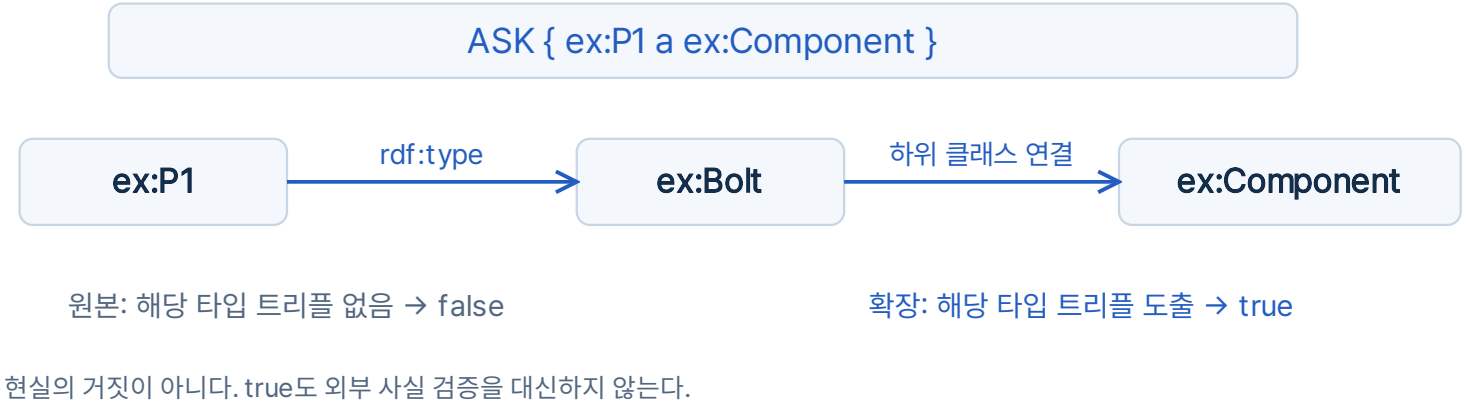
조회 결과	원본	RDFS	OWL-RL
P1 a Component	없음	있음	있음
A1 a Assembly	없음	있음	있음
A1 hasPart P1	없음	없음	있음
제조사 기록	없음	없음	없음

- 입력: P1의 볼트 타입, A1 장착 관계, 클래스·역관계 정의
- 출력: 부품 타입·조립품 타입·역관계의 존재 여부
- 버튼은 실제 저장 결과를 비교하며 새 Python 실행을 시작하지 않음

조회 실패와 현실의 거짓

> 단순 ASK의 false는 선택한 그래프에서 해당 패턴을 찾지 못했다는 뜻입니다. 현실에서 거짓이라는 판정과 구분합니다.

ASK 결과는 선택한 그래프의 패턴 일치 여부



false는 현실의 거짓이 아니다. true도 외부 사실 검증을 대신하지 않는다.

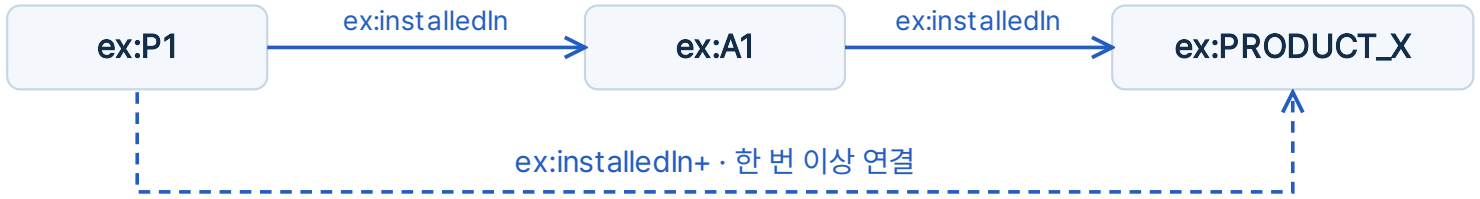
```
ASK { ex:P1 a ex:Component }  # P1의 Component 타입이 조회되는가
```

- 원본 false → 확장본 true: 새 사실을 외부에서 확인한 것이 아니라 정의를 적용한 결과

다단계 관계의 고정점 계산

> 여러 관계를 합쳐 도달 가능한 대상 쌍을 계산합니다. 가능한 모든 경로를 나열하는 작업과는 다릅니다.

경로 탐색은 연결된 시작점 · 끝점 쌍을 반환



반환 예: (P1, A1), (P1, PRODUCT_X) · 통과한 노드 목록이나 경로 수를 뜻하지 않는다.

```
edge(P1, A1), edge(A1, PRODUCT_X)
reach(X, Y) ← edge(X, Y)                    # 직접 연결도 도달
reach(X, Z) ← reach(X, Y), edge(Y, Z)       # 도달 관계를 연장
```

- 실제 Python 집합 계산의 추가 결과: `(P1, PRODUCT_X)`

SKOS 용어 분류와 OWL 클래스

> 용어집의 상하 개념, 클래스 포함 관계, 개체의 타입은 서로 다른 관계입니다. 같은 "볼트"라는 말도 모델링 역할을 구분합니다.

개념 분류와 클래스 분류의 서로 다른 연결



skos:broader와 rdfs:subClassOf는 다른 의미다. broader 자체를 추이적 속성으로 읽지 않는다.

- SKOS: 용어·라벨·분류 체계의 연결
- OWL/RDFS: 개체가 어떤 클래스에 속하는지와 논리적 관계 정의

출처: SKOS Primer · SKOS Reference

SHACL의 입력과 검사 결과

> SHACL 통과는 선언한 조건의 충족입니다. 제조사 정보가 현실에서 참이라는 인증은 아닙니다.



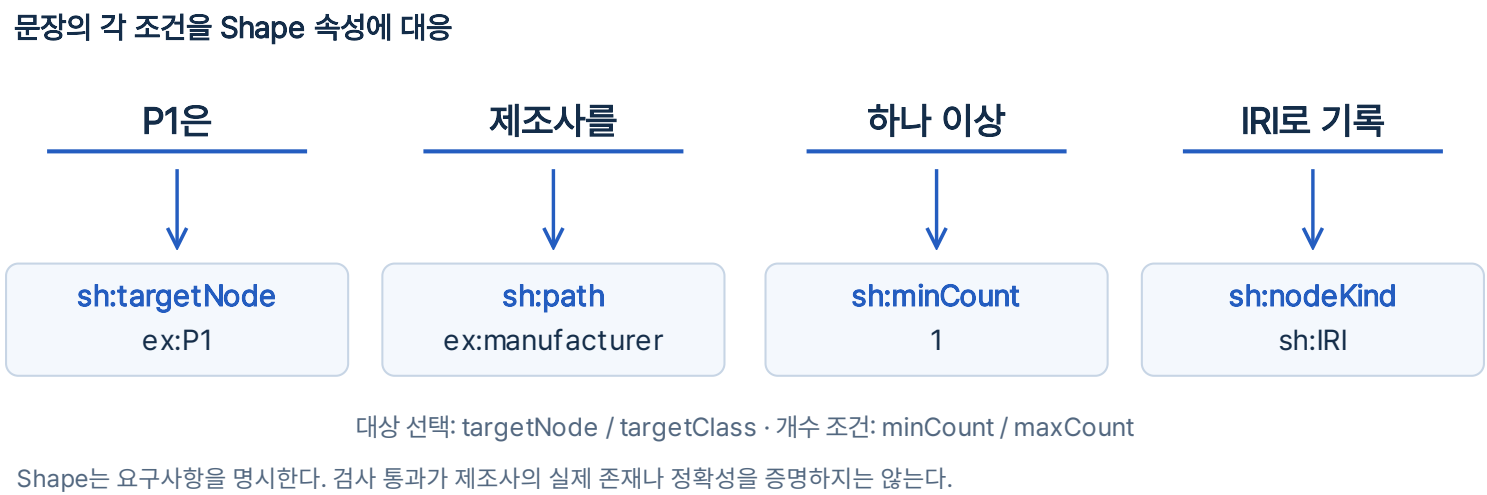
SHACL: Shapes Constraint Language. **Shape**: 검사 대상과 조건을 묶은 구조.

- Data Graph: 검사할 부품 기록
- Shapes Graph: “P1에 제조사 IRI가 하나 이상 있어야 한다”
- Validation Report: 어떤 개체·속성이 어떤 조건을 어겼는지 기록

출처: SHACL

업무 문장의 Shape 변환

> Shape 작성은 요구 문장을 검사 대상·속성·개수·값 종류로 나누는 과정입니다. 뜻이 맞는지는 정상·위반·결측 반례로 확인합니다.



- `targetNode`는 특정 개체, `targetClass`는 클래스의 대상 개체를 선택
- 문법이 맞는 Shape도 잘못된 업무 조건을 검사할 수 있음

LLM으로 RDF·Shape 초안 작성

① 문서와 요구 조건을 프롬프트로 전달

입력

문서: “볼트 P1은 조립품 A1에 장착된다.”

업무 조건: “P1의 제조사를 IRI로 하나 이상 기록해야 한다.”

프롬프트 예시

문서의 사실을 `data.ttl`로, 업무 조건을 SHACL 문법의 `shapes.ttl`로 작성해 줘.

타입은 `Bolt`, 관계는 `installedIn`과 `manufacturer`를 사용해. 제조사 값은 문서에 없으니 data에는 추가하지 마.

② RDF와 Shape 초안 · 기대 출력 예시

```
# data.ttl
ex:P1 a ex:Bolt ;
  ex:installedIn ex:A1 .

# shapes.ttl
ex:ManufacturerShape a sh:NodeShape ;
  sh:targetNode ex:P1 ;
  sh:property [
    sh:path ex:manufacturer ;
    sh:minCount 1 ;
    sh:nodeKind sh:IRI
  ] .
```

③ 파일 저장 → RDF 파싱 → pySHACL 실행
다음 실습에서 이 데이터와 조건을 직접 검사한다.

발표자가 작성한 대화·출력 예시 · 실제 LLM 호출 아님 · 접두사는 실습 노트북에 포함

직접 스키마 작성과 표 데이터 변환

> 클래스·속성·IRI를 먼저 정하고 원천의 열을 그 정의에 연결합니다. 편집 도구·변환 도구·검증 도구의 역할을 나누면 작업을 재현하기 쉽습니다.

스키마 먼저 · 편집 작업의 개념도

클래스 계층 / 속성 / 개체

```
Component
└ Fastener
   └ Bolt

installedIn: Component → Assembly
P1: Bolt → installedIn → A1
```

Protégé / WebProtégé에서 클래스·속성·공리를 편집하고 저장한다.

행 데이터에서 그래프까지

작업	도구·산출물
CSV 정리·ID 맞추기	OpenRefine
열 → IRI·관계 매핑	RDF Transform 확장
반복 가능한 변환	Python + RDFLib
RDF·Shape 직접 작성	텍스트 편집기 + Turtle
조건 검사	pySHACL + 보고서

UI는 기능 설명용 재구성이다. 설치·실행 캡처가 아니며 RDF 확장 호환성은 확인해야 한다.

출처: Protégé / WebProtégé · OpenRefine 확장 목록 · RDFLib

노트북 실습 — 데이터와 Shape 준비

- P1의 기록과 제조사 필수 조건을 각각 RDF 그래프로 읽는다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [2] data

```
DATA = """ex:P1 a ex:Bolt ;
    ex:installedIn ex:A1 ."""
data = Graph().parse(data=PREFIX + DATA, format="turtle")
print(f"데이터: {len(data)}개 트리플")
```

Out [2]

```
데이터: 2개 트리플
```

In [3] shape

```
SHAPE = """ex:ManufacturerShape a sh:NodeShape ;
    # 이 검사는 P1의 제조사 기록만 확인한다.
    sh:targetNode ex:P1 ;
    sh:property [
        sh:path ex:manufacturer ;
        # 값이 하나 이상이고 IRI인지 검사한다.
        sh:minCount 1 ;
        sh:nodeKind sh:IRI ;
        sh:message "제조사 식별자를 하나 이상 기록하세요."
    ] ."""
shapes = Graph().parse(data=PREFIX + SHAPE, format="turtle")
print(f"Shape: {len(shapes)}개 트리플")
```

Out [3]

```
Shape: 7개 트리플
```

▶ 셀의 실제 저장 출력 표시

노트북 실습 — 검사와 보고서 확인

- `conforms`는 조건 충족 여부, `report`는 위반 근거, `report_text`는 텍스트 보고서다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [4] validate

```
conforms, report, report_text = validate(
    data, shacl_graph=shapes, inference="none")
print("conforms =", bool(conforms))
```

Out [4]

```
conforms = False
```

In [5] report

```
for result in report.subjects(RDF.type, SH.ValidationResult):
    print("대상:", report.value(result,
SH.focusNode).n3(data.namespace_manager))
    print("속성:", report.value(result,
SH.resultPath).n3(data.namespace_manager))
    print("조건:", str(report.value(result,
SH.sourceConstraintComponent)).split("#")[-1])
    print("메시지:", report.value(result, SH.resultMessage))
```

Out [5]

```
대상: ex:P1
속성: ex:manufacturer
조건: MinCountConstraintComponent
메시지: 제조사 식별자를 하나 이상 기록하세요.
```

▶ 셀의 실제 저장 출력 표시

제조사 입력별 검사 결과

> 네 번째 입력의 제조사 IRI도 형식·개수 조건은 통과합니다. **그 회사가 실제 제조사인지는 확인하지 않았습니다.**

저장된 제조사 Shape 검사 · 네 입력

입력	conforms	위반 조건	현실의 사실 확인
ex:P1 ex:manufacturer ex:MakerA .	true	없음	미확인
(제조사 기록 없음)	false	MinCountConstraintComponent	미확인
ex:P1 ex:manufacturer "MakerA" .	false	NodeKindConstraintComponent	미확인
ex:P1 ex:manufacturer ex:UnverifiedMaker .	true	없음	미확인

검사 범위와 추론 설정

> 두 검사에서 제조사 기록 부재는 남습니다. 이 예제는 `sh:class` 대신 명시적 타입 값의 존재를 검사해 입력 범위 차이를 보여 줍니다.

저장된 두 Shape 검사 · 모든 호출은 inference=none

검사 대상	Component 타입	제조사	위반 수
원본	미충족	미충족	2
RDFS 확장	충족	미충족	1
OWL-RL 확장	충족	미충족	1

```
# 두 번째 Shape: Component 타입 트리플이 실제로 있어야 함
sh:path rdf:type ; sh:hasValue ex:Component .
```

발주·승인·조직의 연결 키

> 테이블 이름보다 연결 키를 먼저 봅니다. 이 예제는 요청 부서와 검토자 부서가 같은 본부로 연결되는 합성 입력입니다.

① 발주

request_id	item_id	req_dept_id
REQ_2026_01	ITEM_101	DEPT_REQ

② 승인 기록

request_id	reviewer_id	review_role
REQ_2026_01	USER_PRIMARY	Primary
REQ_2026_01	USER_AUDIT	Audit

③ 검토자

reviewer_id	belongs_to_dept_id	qualification_status
USER_PRIMARY	DEPT_REV1	Qualified
USER_AUDIT	DEPT_REV2	Qualified

④ 조직

dept_id	parent_division_id
DEPT_REQ	DIV_SALES
DEPT_REV1	DIV_SALES
DEPT_REV2	DIV_AUDIT

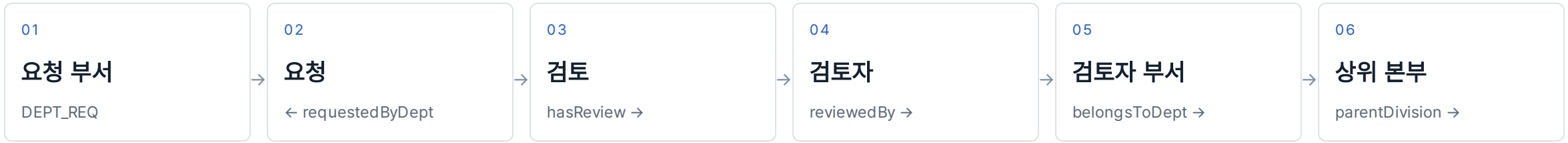
⑤ 계약: ITEM_101 · Active · 할인 상한 0.15 / 발주 할인 0.12 · 순금액 85000 — 저장된 JSON 발췌

- 목적: 승인 표시가 있어도 R3 본부 분리 조건을 위반하는지 확인

R0–R3의 업무 조건

> R3의 SHACL 경로는 역방향 1개와 순방향 4개입니다. 전 라운드에서 같은 RDF 투영기를 사용합니다.

라운드	추가하는 조건
R0	Active 계약, 할인 상한, 순금액 ≤ 100000, 적격 Primary 승인
R1	달성 마진율 ≥ 최소 마진율
R2	적격 Audit 승인도 요구
R3	요청 부서와 검토자 부서의 상위 본부 분리



출처: 통합 진단의 정정 기록

본부 충돌 예제의 실행 코드

> 세 구현은 같은 입력과 R3 정책을 사용합니다. 이 사례에서는 모두 본부 충돌을 이유로 blocked를 반환했습니다.

- 목적: 합성 입력 생성 → 세 구현 판정 → 보고서와 프롬프트 구성

```
tables = make_valid_base_tables(rev1_div='DIV_SALES')  # 본부가 겹치는 입력
decisions = {  # 동일한 R3 정책을 세 구현에서 검사
    'Python': evaluate_arm_p_python(tables, 'R3'),  # Python 함수
    'SQLite': evaluate_arm_s_sqlite(tables, 'R3'),  # SQL 조회·판정
    'SHACL': evaluate_arm_g_shacl(tables, 'R3'),    # RDF·Shape 검사
}
report = extract_shacl_llm_report(tables, 'R3')  # 보고서를 JSON 필드로 정리
prompts = build_llm_prompt_modes(tables, round_id='R3')  # 입력문만 생성
```

실제 저장 출력	판정	위반
Python = SQLite = SHACL	blocked	division-separation

실행 코드에 설명 주석만 추가 · 전체 원본은 자료 창의 seminar_examples.py

검증 보고서의 근거 필드

> 보고서는 "왜 차단됐는가"를 개체·관계 경로·값으로 나눕니다. `conforms` 와 최종 업무 판정은 같은 필드가 아닙니다.

- 목적: 실제 보고서의 필드를 설명에 사용할 근거로 읽기

실제 출력 발췌 · JSON

```
{
  "focus_node": "ex:departments/DEPT_REQ",
  "value_node": "ex:divisions/DIV_SALES",
  "constraint_component": "sh:DisjointConstraintComponent"
}
```

각 필드의 역할

필드	읽는 법
focus_node	요청 부서 DEPT_REQ가 검사 대상
result_path	요청→검토→검토자→부서→본부를 따라간 경로
value_node	겹치는 본부 DIV_SALES
constraint_component	두 값 집합의 겹침을 금지하는 Disjoint 검사

- 경로: 요청 부서 ← 요청 → 검토 → 검토자 → 소속 부서 → 본부
- 표준 RDF 보고서를 이 JSON으로 변환하는 코드는 저장소에서 직접 구현

LLM 입력으로 전달할 검사 근거

> 검증기의 출력은 LLM이 사용할 설명 재료가 됩니다. 여기서는 프롬프트를 만들었으며 LLM 응답 정확도는 측정하지 않았습니다.

입력 자료의 구성 비교

구성	LLM 앞에 놓는 자료
M0	질문 + 규정 + 원천 테이블
M1	질문 + 테이블 + 함수 판정·규칙명
M2	질문 + 판정·개체·경로·값 보고서

목적: 모델이 설명에 사용할 근거가 어떤 형태로 전달되는지 확인.

M2 입력문 일부 · 구조 설명용

```
질문: 이 요청을 승인할 수 있는가?

검사 결과: blocked
위반 규칙: division-separation
검사 대상: DEPT_REQ
충돌 값: DIV_SALES

지시: 위 근거를 사용해 설명하라.
```

이 화면은 프롬프트 구성이다.
이번 진단에서는 모델 응답을 받지 않았다.

- 같은 보고서를 만드는 Python·SQL 조건과의 모델 비교는 별도 과제

출처: 실행 기록

반례와 근거 상태 진단

> 반례는 잘못 옮긴 조건을 찾기 위한 진단입니다. 근거 상태 비교에서는 검색 누락과 원천 부재를 구분합니다.

1단계 · 잘못 옮긴 규칙 찾기

입력 24항목 × 정상·경계 위반·무관 변화·결측 반례

조건 의도적으로 틀린 함수 / 기준 구현

출력 네 반례에서 모두 기대 판정과 일치한 항목

오류 함수	기준 구현
0/24	24/24

2단계 · 부족한 근거 처리

입력 동일 업무 정책, 여섯 근거 상태 × 60개 ID

처리 unknown → 전체 원천을 한 번 재평가

출력 누락이 채워지면 재판정, 원천에도 없으면 보류

S2 계약 판정 일치	외부 합성 진실 오류
360/360	사실 치환 60/60

- 고정 함수를 오프라인으로 비교했다. 자연어를 자동 이해한 성능이 아님

출처: 실행 기록

규칙 추가와 보고서 필드 진단

> 규칙이 늘어날 때 판정이 같은지, 설명에 필요한 필드가 채워지는지를 각각 확인했습니다.

3단계 · 규칙 추가 후 판정 비교

입력 같은 테이블의 6패턴

변경 R0 → 마진 R1 → Audit R2 → 본부 분리 R3

출력 Python·SQLite·SHACL의 반환값 비교

960 / 960 일치

6패턴 × 40반복 × 4라운드. 실제 사람의 수정 시간을 잰 결과는 아니다.

4단계 · 보고서의 설명 재료

입력 4위반 패턴 × 60반복

출력 프롬프트에 채워진 필드 수

구성	판정	개체·경로	값
M0	0	0 · 0	0
M1	240	0 · 0	0
M2	240	240 · 240	180

각 분모 240. 기수성 위반 60건에는 충돌 값이 없다.

- 출력 일치·필드 개수로 유지보수 시간이나 LLM 설명 정확도를 대신 평가할 수 없음

출처: 실행 기록

계산된 근거와 LLM 판단

> 별도 선행 실험에서는 원시 근거와 계산된 보고서를 제공했을 때 LLM의 계약 일치를 비교했습니다.

비교한 처리 흐름

① 질문 + 원시 근거 → **LLM 직접 판단**

② 같은 정책의 계산 결과 + 근거 → **LLM 판단**

목적: 미리 계산한 관계·조건 정보가 계약에 맞는 판단에 도움을 주는지 확인.

189개 완료 묶음: 첫 반복 120 + 두 번째 반복 69.

저장된 관측값

지표	직접	판단 전 검사
계약 일치	124/189	179/189
잘못된 긍정	15	0
불필요한 부정 / 보류	26 / 3	9 / 1
무관 정보 후 판정 변화	21/77	1/77

계약 일치는 외부 사실 정확도와 구분한다.

- 이번 M0·M1·M2 진단 결과가 아니다. 계산된 정보를 제공한 효과와 RDF 고유 효과를 구분

출처: 실행 기록

동일 의미의 Python·OWL-RL 비교

> 기존 추론 연구에서는 동일한 정의를 받은 Python과 OWL-RL이 같은 결론을 내리는지 확인했습니다.

입력·조건·출력

입력 64개 합성 그래프의 사실과 동일한 공리

질문 직접 쓰지 않은 타입·역관계·도달 관계도 찾는가?

조건 명시 조회 / JSON·Python 규칙 / RDF·OWL-RL

출력 640질의의 결론과 배타 클래스 충돌 증거

같은 의미를 받은 두 구현

지표	명시 조회	Python	OWL-RL
질의 일치	504/640	640/640	640/640
놓친 도출 답	136	0	0
충돌 그래프 탐지	0/32	32/32	32/32

추론을 추가한 효과와 RDF 형식의 효과를 분리한다. LLM 호출 없음.

- 제한된 양의 규칙과 지정 충돌 증거의 결과다. 전체 OWL의 완전성·실제 업무 정확도는 미평가

출처: 실행 기록

실제 GLEIF의 관계·등록 상태

> 실제 데이터에는 관계의 유무뿐 아니라 관계 종류·등록 상태·유효 시점이 함께 기록됩니다. 기록된 사실과 의사결정 채택 기준을 분리해야 합니다.

실제 GLEIF 저장 원천 · RR 10행 발췌

필드	값
StartNode.NodeID	010PWNH4K3BLIC3I7R03
EndNode.NodeID	549300COKYB5EGSU1838
RelationshipType	IS_DIRECTLY_CONSOLIDATED_BY
RelationshipStatus	ACTIVE
RegistrationStatus	LAPSED

레슨런 · 기록과 채택 기준 분리

기록 사실 직접 회계 연결 관계가 원천에 있다.

상태 의미 관계 ACTIVE와 등록 LAPSED는 다른 필드다.

이번 정책 PUBLISHED만 채택하므로 판단은 보류했다.

후속 화면 "관계가 없음" 대신 "보고된 관계는 있으나 채택 기준 미충족"으로 표시한다.

출처: 실제 GLEIF 예시와 정책 선택

부품 인증 만료의 영향 범위

> 후속 설계안입니다. 재검토 후보 도출은 물리적 고장 예측이나 제품 안전성 판정이 아닙니다.



"이 부품의 인증서가 만료됐습니다. **어떤 완제품을 재검토해야 하나요?**"

- 사용 관계: 인증서 → 부품 → 조립품 → 완제품
- 적용 조건: 부품 유형 → 상위 유형 → 적용 규정
- 판정: 해당 시점·규정에서 재검토 후보인지 확인

원천 부재와 검색 누락의 차이

> 추가 관계나 정의를 제거했을 때 답이 달라지는 대응 사례가 필요합니다. 홉 수만 늘리는 것으로 온톨로지의 효과가 입증되지는 않습니다.

원천에도 관계가 없는 경우

원천 BOM	parent	child
b1	A1	P1
—	PRODUCT_X	미기록

검색 반환	parent	child
b1	A1	P1

새 검색만으로 빠진 관계를 만들 수 없다.

출력: 완제품 범위 확인 불가

원천에는 있지만 검색에서 누락

원천 BOM	parent	child
b1	A1	P1
b2	PRODUCT_X	A1

검색 반환	parent	child
b1	A1	P1

반환 범위를 보완하면 b2를 찾을 수 있다.

출력: 보완 후 PRODUCT_X 후보

작성한 설명용 데이터. 원천 스냅샷을 따로 보존해야 두 실패를 구분할 수 있다.

- 같은 질문: “P1과 연결된 완제품의 재검토 범위는?”
- 상위 유형 규정 적용은 명시적 정책으로 선언; subclass만으로 임의 규정이 복사되지 않음

후속 가설과 평가 지표

> 모두 **제안 가설**입니다. 완료된 연구의 사전등록 가설이나 새 측정 결과로 제시하지 않습니다.

제안 가설	통제할 것	관측할 것
관계 계산이 LLM의 누락·오연결을 줄이는가	같은 질문·입력·정책	후보 재현율·정밀도, 잘못된 확답
공유 의미 정의가 질문 간 불일치를 줄이는가	동일 버전·정보 접근	질문 간 모순, 변경 후 회귀 오류
구현 차이가 변경·근거 추적 비용에 나타나는가	동등한 공리·규칙	실제 수정·테스트·추적 작업량
불완전한 근거에서 균형 있게 보류하는가	답변 가능 여부	잘못된 확답 + 불필요한 보류

동일 의미·정보를 받은 비교 조건

> 순서·도구 호출은 DAG로 구성할 수 있습니다. 모든 조합을 한꺼번에 돌리기보다 하나의 효과씩 분리합니다.

공통 계약

같은 분류·관계·정책
같은 입력과 증거 범위

→

각 구현

Python / SQL / JSON
RDF / OWL / SHACL

→

별도 평가

결론 일치·누락·근거
변경 비용은 따로 측정

대조	분리하려는 차이
명시 조회 ↔ 같은 정책의 관계 계산	계산을 추가한 효과
Python·재귀 SQL ↔ RDF·규칙/OWL	같은 의미의 구현 방식
개별 설정 ↔ 공유 JSON/스키마 ↔ 공유 온톨로지	의미·버전 공유 방식
동일 보고서 유무를 바꾼 LLM	계산된 근거를 제공한 효과

여섯 가지 근거 상태

> 같은 질문에 서로 다른 근거 상태를 만들어 잘못된 확답과 불필요한 보류를 함께 확인합니다. 아래는 후속 데이터셋의 설명용 설계입니다.

상태	작은 레코드 예	확인할 출력
충분한 근거	b1: A1→P1 / b2: X→A1	X와 근거 b1,b2
원천 부재	source에 b2 없음	확인 불가
검색 누락	source에는 b2 / retrieved에는 없음	검색 보완 여부
시점·버전 불일치	valid_to=2025 / 질문=2026	현재 적용 여부
자료 충돌	v1: P1→A1 / v2: P1→A2	선택 근거 또는 보류
일관된 거짓 입력	P1→A1 기록, 현장 증거는 다름	검증 통과와 외부 오류 분리

- 원천·검색 반환·질문 시점·평가 기준을 별도로 저장
- 숨은 평가 기준으로 실행 답을 수정하지 않음

원천·합성·평가 자료의 구분

> 실제 스키마를 참고해 만든 거래 데이터는 합성 데이터입니다. 사용한 코호트를 새 held-out 자료처럼 재사용하지 않습니다.

실제 원천 자료

출처·버전·해시 보존
실제 확인한 관계만 표시

작성한 규칙·사례

합성 거래·관계·정책
공개 스키마 참고도 합성

평가 자료

개발·평가 사용 이력 기록
정답은 실행 중 제공 안 함

- 원천: 파일·해시·출처·사용 조건·시점·버전
- 작성 부분: 합성 관계와 가정한 규칙을 별도 표시
- 실행 근거: 실제 검색 반환과 추론·검증 설정 저장
- 평가 기준: 계약 일치와 외부 사실 판단을 별도 기록

관계 변경과 스키마 관리

> 관계는 바뀝니다. 안정된 식별자, 입력 매핑, 의미 변경, 시점·버전을 구분해 관리하는 범위를 작게 시작할 필요가 있습니다.

속성명 변경과 의미 변경의 분리

변경 예	권장 처리
maker → manufacturer	뜻이 같다면 안정 IRI 유지 + 입력 매핑 변경
assembledIn → installedIn	동일 의미인지 먼저 검토; 이름만 보고 통합 금지
소유 → 운용	다른 역할이면 별도 속성·정의
설치 위치의 시점 변경	관계 레코드에 유효 기간·출처·버전 저장

작은 공통 계약 + 도메인 확장

```
원천 필드 / 버전
      ↓ 매핑
안정된 관계 IRI + 역할 정의
      ↓
질문별 규칙·Shape + 회귀 사례
      ↓
영향받는 조회·소비처 확인
```

모든 관계를 미리 표준화하기보다 반복 질문에 필요한 범위를 우선 관리한다. 유지보수 비용은 남는다.

- 운영 설계 제안: 영향받는 질의·Shape·소비처를 회귀 사례로 확인
- 실제 유지보수 시간과 장기 비용 절감은 아직 측정하지 않음

근거를 포함한 검토 화면

> 최종 화면에는 답뿐 아니라 적용한 규칙과 근거가 함께 보여야 합니다. 다음 데모 개발은 이 구조를 실제 사용자 흐름으로 연결하는 작업입니다.

발주 검토 · 실제 저장 결과로 구성한 발표용 화면

질문과 입력

REQ_2026_01을 승인할 수 있는가?

조건	저장된 입력
계약 / 할인	Active / 0.12 ≤ 0.15
검토자	USER_PRIMARY · Qualified
요청 본부	DIV_SALES
검토자 본부	DIV_SALES

검사 결과와 근거

blocked · 승인 조건 미충족

R3 본부 분리 조건에서 동일 본부가 확인됐다.

보고서 필드	값
focus_node	DEPT_REQ
value_node	DIV_SALES
violations	division-separation

설명은 발표자가 작성했다. 실제 LLM 응답이나 운영 서비스 화면은 아니다.

자료 창에서 project-input.json · project-report.json · 실제 호출 코드 열기